



SUMMARY AND ADVANCED TOPICS

Summer School on HPC Driving AI

June 20, 2026 | Prateek Chawla | Jülich Supercomputing Center
(Slides originally by Andreas Herten, Jülich Supercomputing Center)

Overview

Summary

1L: Intro & JUPITER

2L: MPI-Distributed GPU Computing

4L: Performance/Debugging Tools

5L: Optimization Techniques

7L: NCCL, NVSHMEM

9L: CUDA Graphs, Device-Initiated
NVSHMEM

More: Other Languages/Models

OpenACC, OpenMP; Kokkos
Python

More: In-Network Computing

Concept
Libraries

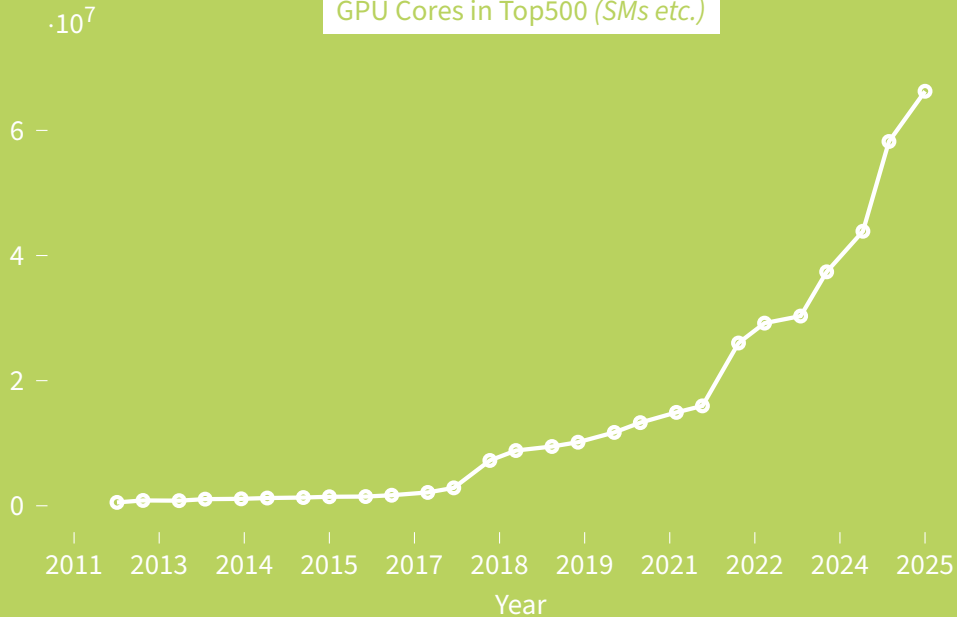
Other Vendors

Summary, Conclusion

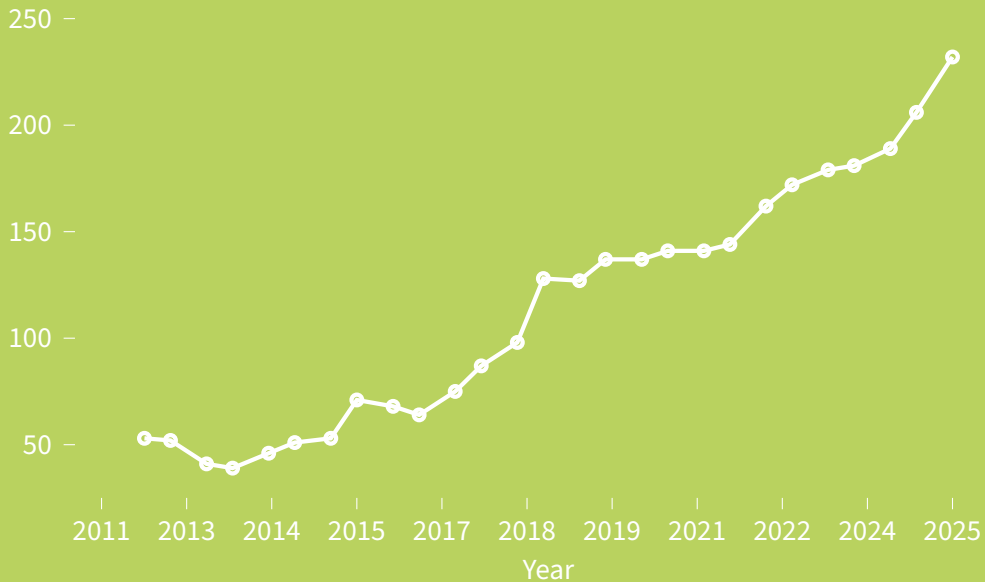
Summary

1L: Intro & JUPITER

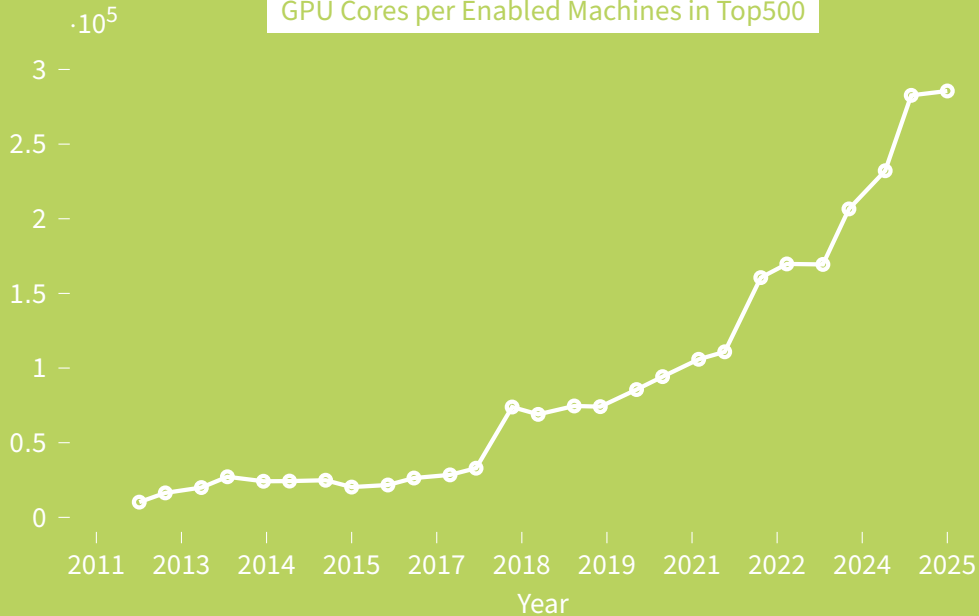
GPU Cores in Top500 (*SMs etc.*)



GPU-enabled Machines in Top500

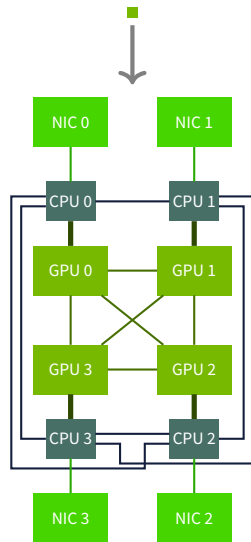


GPU Cores per Enabled Machines in Top500



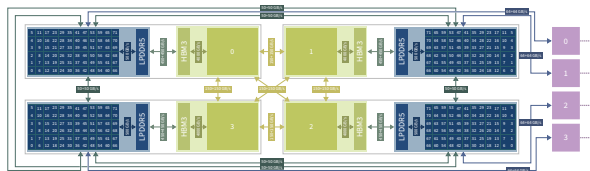
JUPITER

- At Jülich Supercomputing Center, Germany; procured by EuroHPC JU
 - First European Exascale system
 - **Booster**, Cluster
 - HPL June 2025: 0.8 EFLOP/s
 - **GPU**: 4× NVIDIA H100 *Grace-Hopper* flavor
96 GB memory per GPU
 - **CPU**: 4× NVIDIA Grace, 4× 72 cores; 4× 120 GB LPDDR5X memory
 - **Network**: 4× NVIDIA InfiniBand NDR200, 4 × 25 GB/s
- ➔ ≈6000 nodes, ≈24 000 GPUs, ≈24 000 network devices



Topology

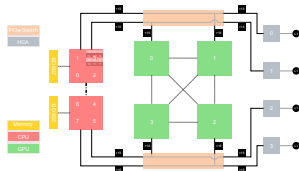
JUPITER Booster



- 4 GH200 superchips → 4 **NUMA** domains
- 1:1:1 affinity of **CPU**, **GPU**, **HCA**; coherent access between **GPU-GPU**, **CPU-GPU**, **CPU-CPU**

No fear: Slurm – and we – take care about affinity!

JUWELS Booster

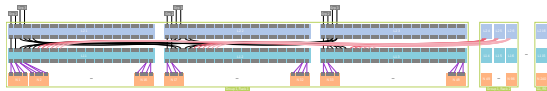


- 2 AMD EPYC CPUs, each 4 chiplets → 8 **NUMA** domains
- Affinity between **GPUs** and some **CPU** chiplets, **HCA**

Some fear: But Slurm still takes care

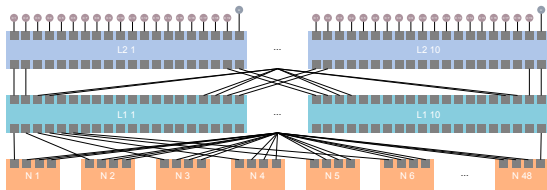
Network

JUPITER Booster



- NVIDIA Mellanox Quantum InfiniBand NDR200 network
- Dragonfly+ topology: 25 sparsely interconnected groups of 240 well-connected nodes (5 racks, each 48 nodes)
- Per group: 15 L1 switches, 16 L2 switches (*rack 3 has one more*)
- 400 Gbit/s links between switches, 200 Gbit/s links from nodes

JUWELS Booster



- NVIDIA Mellanox Quantum InfiniBand HDR200 network
- Dragonfly+ topology: 20 groups of 48 nodes (2 racks, each 24 nodes)
- Per group: 10 L1 switches, 10 L2 switches
- 200 Gbit/s links

Summary

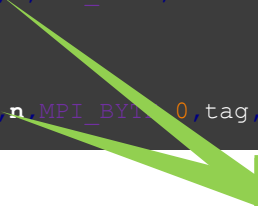
2L: MPI-Distributed GPU Computing



CUDA-aware MPI

CUDA-aware MPI allows you to use Pointers to GPU-Memory as source and destination

```
//MPI rank 0  
MPI_Send(s_buf_d, n, MPI_BYTE, size-1, tag, MPI_COMM_WORLD);  
  
//MPI size-1  
MPI_Recv(r_buf_d, n, MPI_BYTE, 0, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

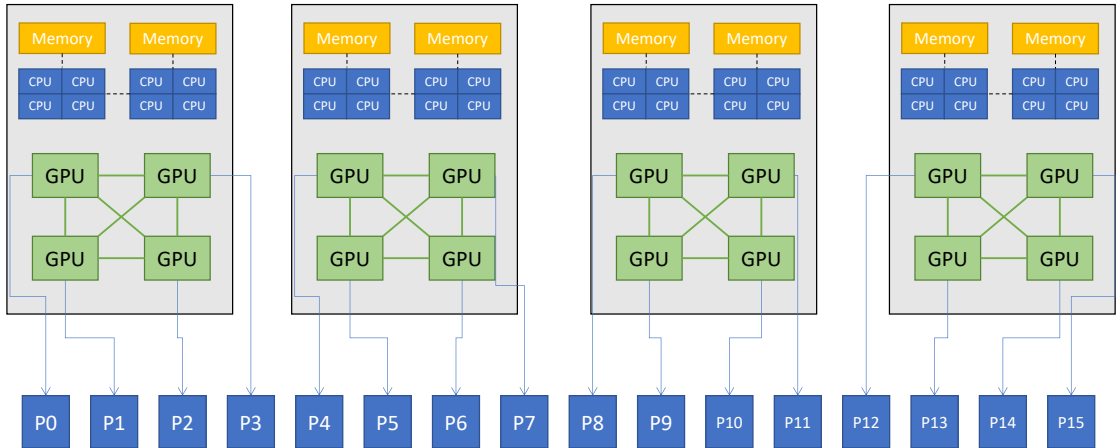


Pointer to
GPU
memory!

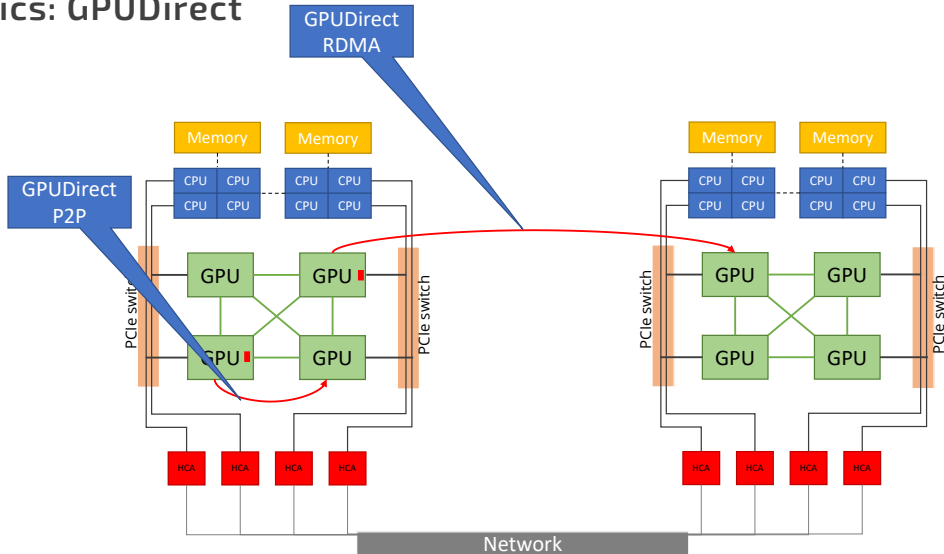


Process Mapping on Multi GPU Systems

One GPU per Process

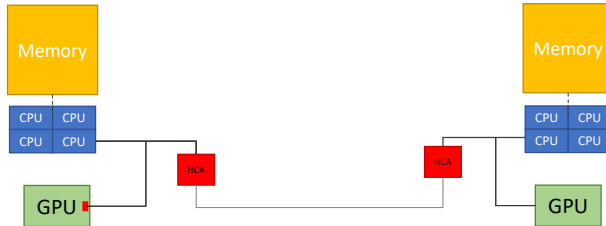


Basics: GPUDirect





CUDA-aware MPI with GPUDirect RDMA

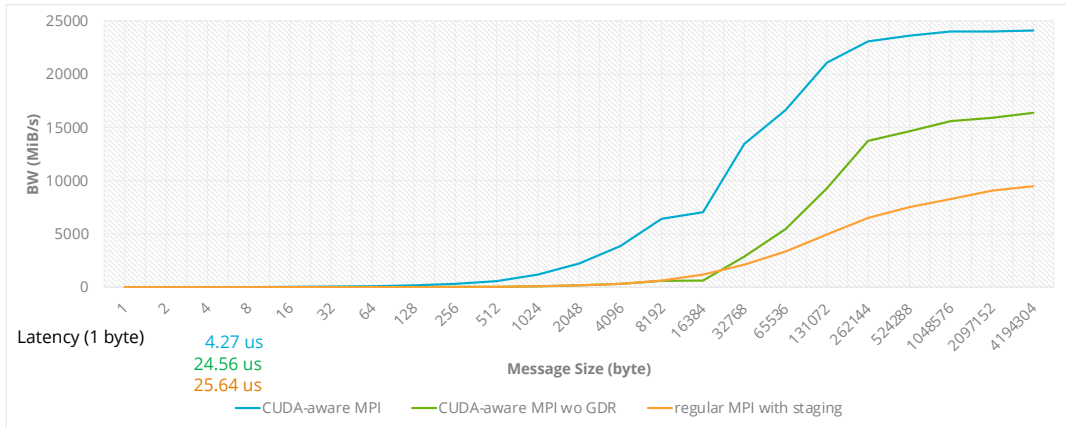


```
MPI_Send(s_buf_d,size,MPI_BYTE,1,tag,MPI_COMM_WORLD);  
MPI_Recv(r_buf_d,size,MPI_BYTE,0,tag,MPI_COMM_WORLD,&stat);
```



Performance Results GPUDirect RDMA

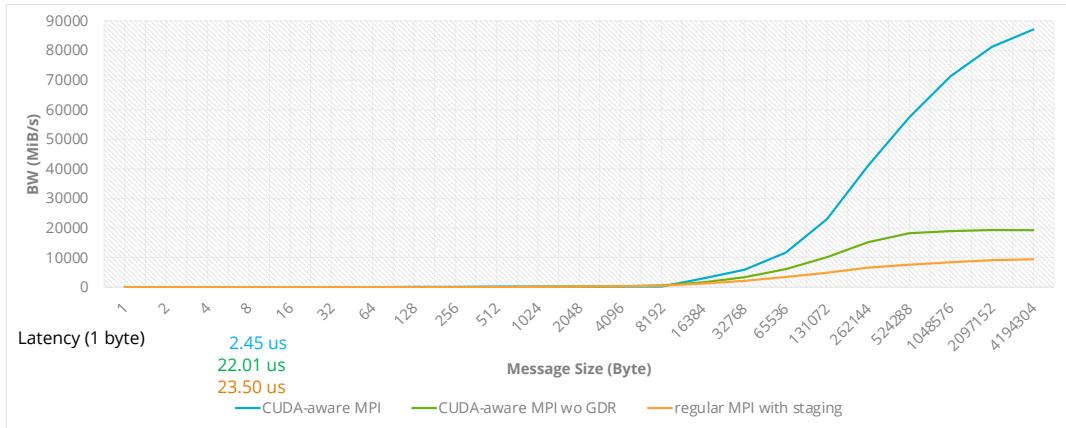
OpenMPI 4.1.0RC1 + UCX 1.9.0 on JUWELS Booster





Performance Results GPUDirect P2P

OpenMPI 4.1.0RC1 + UCX 1.9.0 on JUWELS-Booster



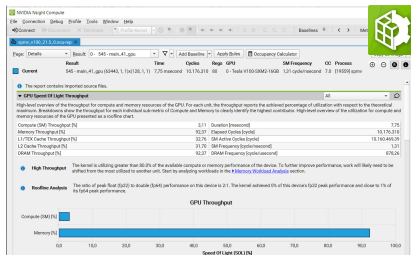
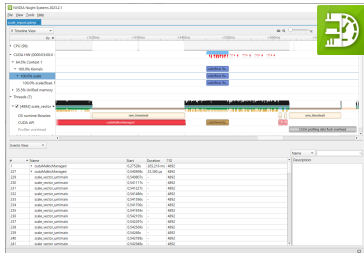
Summary

4L: Performance/Debugging Tools

Where Should I Start Profiling?

And which tool to use?

- Always tradeoff between slightly conflicting goals: Performance vs. Maintainability vs. Effort
- Start with profiling at system-level view → **Nsight Systems**
- Ensure you understand your timeline, and where the GPU is active/inactive
 - What happens only once, what every time step?
 - How does the timeline change for different workloads?
- Take the low-hanging fruits first!
- Don't shy away from kernel optimization but consider effort vs. impact on overall application performance
 - Consider Amdahl's Law: Assume optimized kernel takes 0 s, how large is speedup of entire application?
 - If whole timeline consists of a single kernel, by all means, start profiling and optimizing it → **Nsight Compute**



A first (I)Nsight

Recording and inspecting Nsight Systems profiles

- Record profile using the command line

```
nsys profile --trace=cuda,nvtx,mpi --output=my_report.%q{SLURM_PROCID} ./app
```
- ...and inspect profiling results by opening report in nsys-ui via X-forwarding or locally
- Record profile using the command line, adding `--stats true`

```
nsys profile --stats true --trace=cuda,nvtx,mpi --output=my_report.%q{SLURM_PROCID} ./app
```
- ...and run reports on command line to check validity of profile, and identify important kernels

Several customizable reports based on sqlite + Python included in nsys installation

[4/6] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
70.3	6,305,096	1,001	6,298.8	4,768.0	4,704	1,544,154	48,655.6	jacobi_iteration(float *, const float *, int, int, float)
29.7	2,663,124	1,001	2,660.5	2,592.0	2,496	3,200	130.1	apply_periodic_bc(float *, int, int)

NVIDIA Tools Extension Library

Range-based code annotation API to provide context to tools

- `#include "nvtx3/nvToolsExt.h"`
 - NVTX v3 is header-only
 - C++ and Python APIs
- Fortran: [NVHPC compilers include module](#)
 - Just use `nvtx` and `-lnvhpcwrapnvtx`
 - Other compilers: See blog post linked below
- Use PUSH/POP macros to define ranges conveniently:
 - `PUSH_RANGE(name, color_idx)`
 - <https://developer.nvidia.com/blog/cuda-pro-tip-generate-custom-application-profile-timelines-nvtx/>
- Sprinkle them strategically through code
 - → Use hierarchically: nest ranges

```
int main(int argc, char** argv) {
    PUSH_RANGE("main", 0)
    PUSH_RANGE("init", 1)
    do_initialization();
    POP_RANGE
    /* ... */
    PUSH_RANGE("computation", 2)
    jacobi_kernel<<< /* ... */, compute_stream>>> (...);
    cudaStreamSynchronize(compute_stream);
    POP_RANGE
    /* ... */
    POP_RANGE
}
```

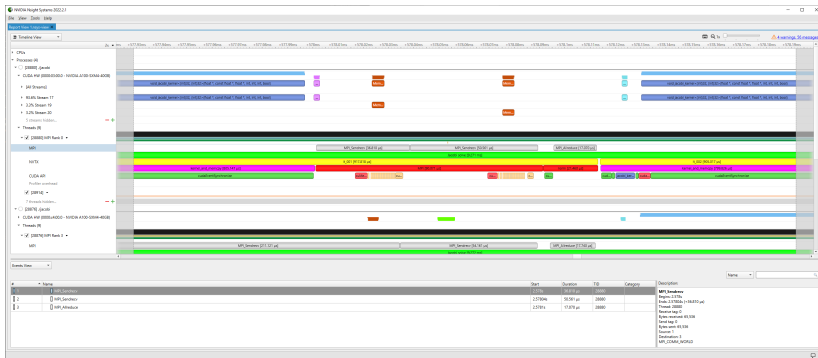
Summary

5L: Optimization Techniques



Multi GPU Jacobi Nsight Systems Timeline

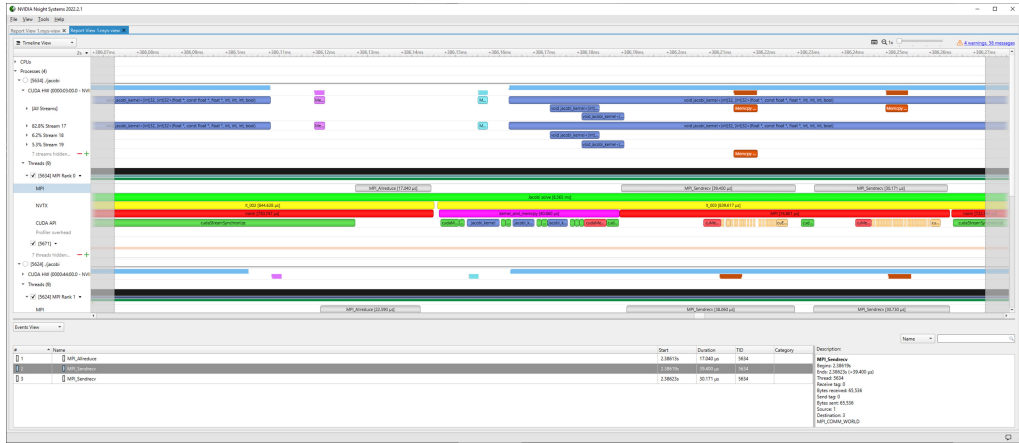
MPI 8 NVIDIA A100 40GB on JUWELS Booster





Multi GPU Jacobi Nsight Systems Timeline

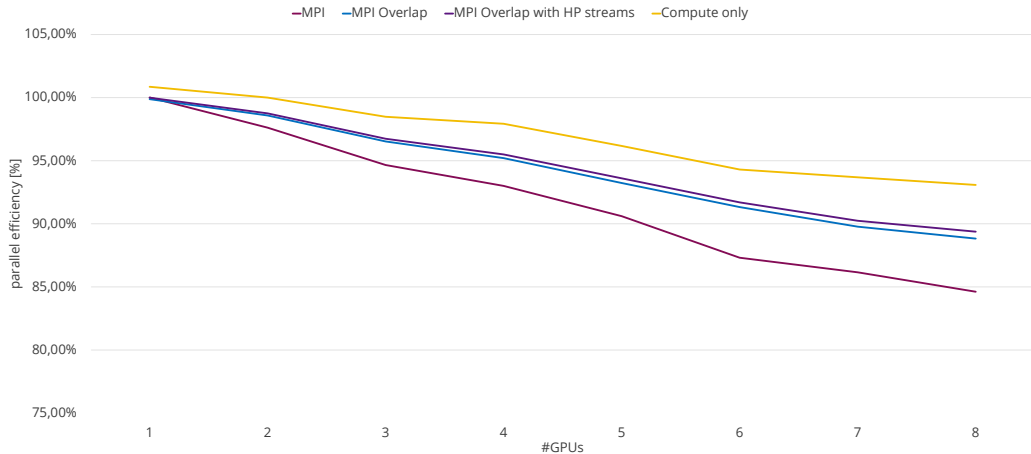
MPI Overlap 8 NVIDIA A100 40GB on JUWELS Booster





Communication + Computation Overlap

ParaStationMPI 5.4.10-1 – JUWELS Booster – NVIDIA A100 40 GB – Jacobi on 17408x17408

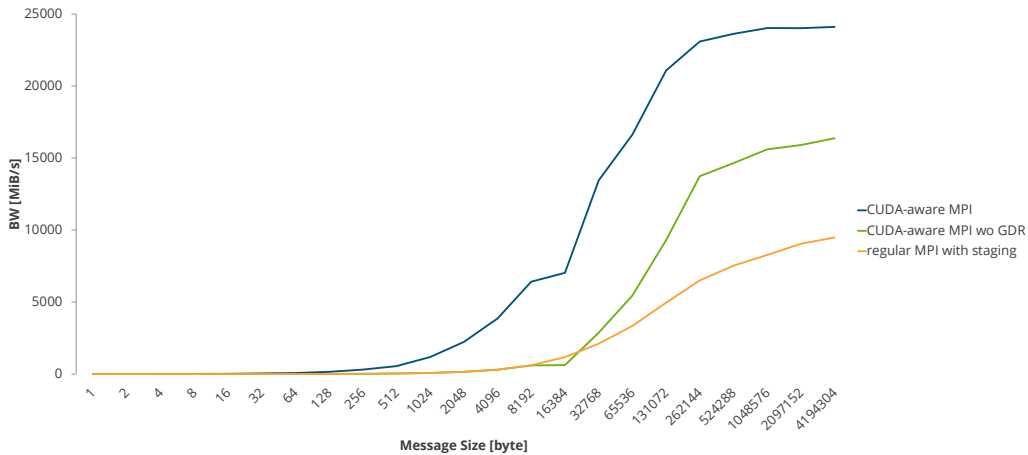


Source: <https://github.com/NVIDIA/multi-gpu-programming-models>
JUWELS Booster: <https://apps.fz-juelich.de/jsc/hps/juwels/booster-overview.html>



Performance Results GPUDirect RDMA

Open MPI 4.1.0RC1 + UCX 1.9.0 on JUWELS Booster



Latency (1 byte) 4.27 us 24.56 us 25.64 us

Summary

7L: NCCL, NVSHMEM



NCCL-API (With MPI) - Initialization

First, we need a NCCL-Communicator for this, we need a NCCL UID

We use MPI-Ranks and size for initialization

```
MPI_Init(&argc,&argv)
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

ncclUniqueId nccl_uid;
if (rank == 0) ncclGetUniqueId(&nccl_uid);
MPI_Bcast(&nccl_uid, sizeof(ncclUniqueId), MPI_BYTE, 0, MPI_COMM_WORLD));

ncclComm_t nccl_comm;
ncclCommInitRank(&nccl_comm, size, nccl_uid, rank);
...
...
ncclCommDestroy(nccl_comm);
MPI_Finalize();
```

Communication Calls

- Send/Recv

Supported for
NCCL 2.8+

```
ncclSend(void* sbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);  
ncclRecv(void* rbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);
```

- Collective Operations

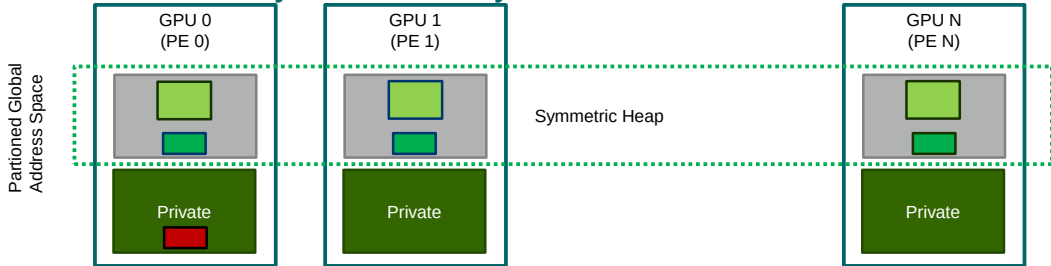
```
ncclAllReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op, ncclComm_t comm, cudaStream_t stream);  
ncclBroadcast(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, int root, ncclComm_t comm, cudaStream_t stream);  
ncclReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op, int root, ncclComm_t comm, cudaStream_t stream);  
ncclReduceScatter(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op, ncclComm_t comm, cudaStream_t stream);  
ncclAllGather(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclComm_t comm, cudaStream_t stream);
```

Jacobi using NCCL and Overlapping Communication and Computation

```
launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, (iy_start + 1), nx, push_stream);
launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_end - 1), iy_end, nx, push_stream);
launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end - 1), nx, compute_stream);

ncclGroupStart();
ncclRecv(a_new, nx, NCCL_REAL_TYPE, top, nccl_comm, push_stream)
ncclSend(a_new + (iy_end - 1) * nx, nx, NCCL_REAL_TYPE, bottom, nccl_comm, push_stream);
ncclRecv(a_new + (iy_end * nx), nx, NCCL_REAL_TYPE, bottom, nccl_comm, push_stream);
ncclSend(a_new + iy_start * nx, nx, NCCL_REAL_TYPE, top, nccl_comm, push_stream);
ncclGroupEnd();
```

NVSHMEM Symmetric Memory Model

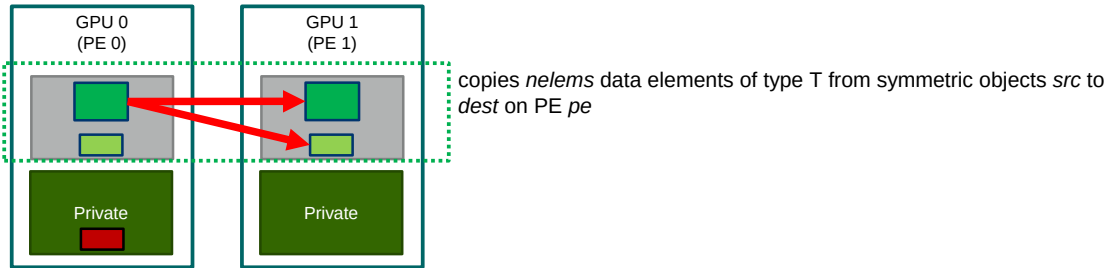


Symmetric objects are allocated collectively with the same size on every PESymmetric memory: `nvshmem_malloc( shared_size);`

Private memory: `cudaMalloc(...)`

Must be the
same on all
PEs

NVSHMEM Host API Put



```
void nvshmem_<T>_put(T* dest, const T* src, size_t nelems, int pe);
void nvshmemx_<T>_put_on_stream(T* dest, const T* src, size_t nelems, int pe,
                                cudaStream_t stream);
```

The x marks
extensions to the
OpenSHMEM API

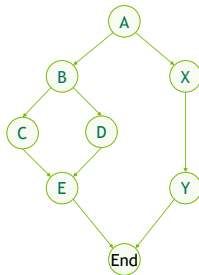
Summary

9L: CUDA Graphs, Device-Initiated NVSHMEM

Asynchronous Task Graphs

A Graph Node is a CUDA Operation

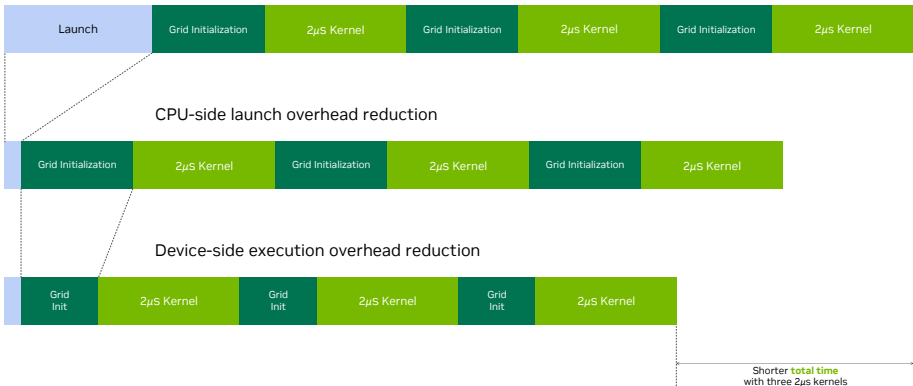
- Sequence of operations (nodes), connected by dependencies
- Operations are one of:
 - Kernel Launch CUDA kernel running on GPU
 - CPU Function Call Callback function on CPU
 - Memcopy/Memset GPU data management
 - Mem Alloc/Free Memory management
 - External Dependency External semaphores/events
 - Sub-Graph Graphs are hierarchical
- Nodes within a graph can also span multiple devices



Where is Performance Coming From?

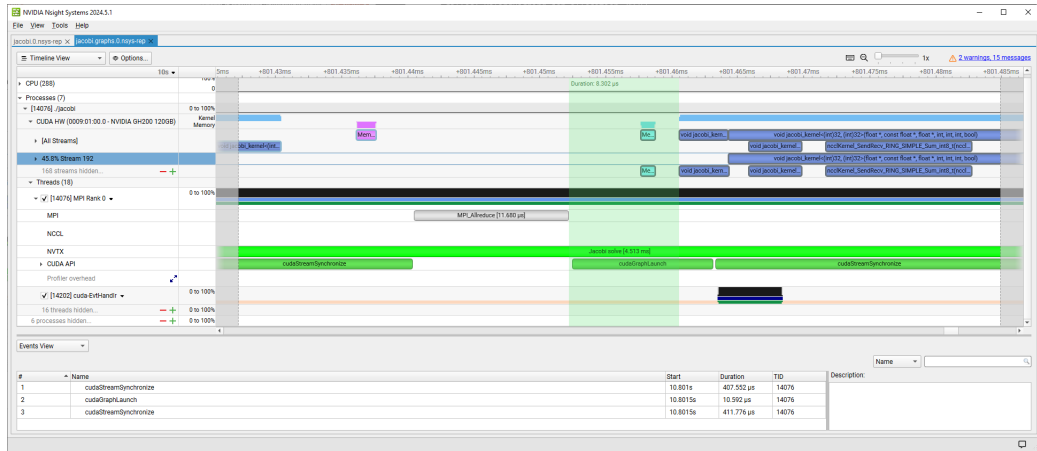
Reducing System Overheads Around Short-Running Kernels

Breakdown of time spent during execution



Multi GPU Jacobi Nsight Systems Timeline

NCCL with CUDA Graphs 4 NVIDIA GH200 120GB on JEDI



Collective Kernel Launch

Ensures progress when using device-side inter-kernel synchronization

NVSHMEM Usage	CUDA Kernel launch
Device-Initiated Communication	Execution config syntax <<<. . .>>> or launch APIs
Device-Initiated Synchronization	<code>nvshmemx_collective_launch</code>

- CUDA's throughput computing model allows (and encourages) grids much larger than a GPU can fit
- Inter-kernel synchronization requires producer and consumer threads to execute concurrently
- Collective launch guarantees co-residency using CUDA cooperative launch and requirement of 1PE/GPU

NVSHMEM API

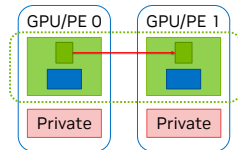
Single Element Put

```
__device__ void nvshmem_TYPENAME_p(TYPE *dest, TYPE value, int pe)
```

- dest [OUT]: Symmetric address of the destination data object.
- value [IN]: The value to be transferred to dest.
- pe [IN]: The number of the remote PE.

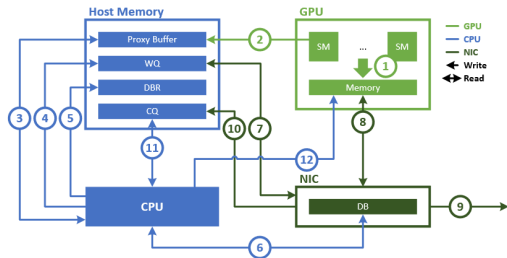
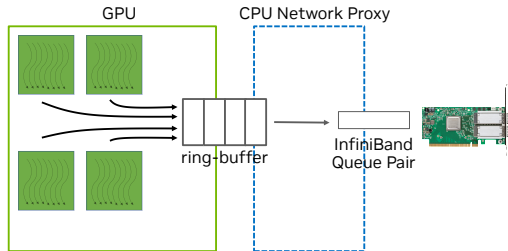
See: <https://docs.nvidia.com/nvshmem/api/gen/api/rma.html#nvshmem-p>

TYPENAME can be: half, float, double, char, schar, short, int, long, longlong, uchar, ushort, ect. (see: <https://docs.nvidia.com/nvshmem/api/gen/api/rma.html#>)



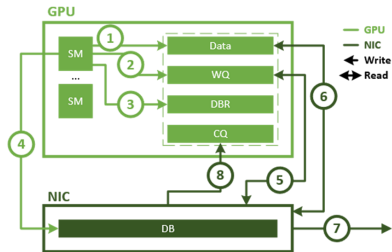
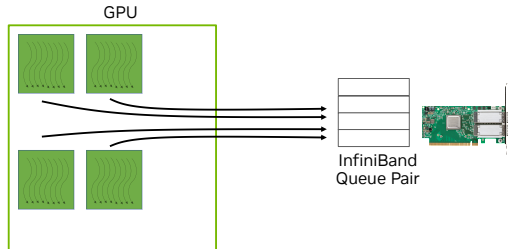
Optimized Inter-Node Communication

- NVSHMEM supports inter-node communication over InfiniBand, RoCE, and UCX (experimental)
- Using GPUDirect RDMA (data plane)
- Reverse offloads network transfers from GPU to the CPU (control plane)
- Ring buffer implementation avoids memory fences when interacting with CPU network proxy



Optimized Inter-Node Communication **Improved**

- **IB GPUDirect Async (IBGDA)** over InfiniBand
- Using GPUDirect RDMA (data plane)
- GPU directly initiates network transfers involving the CPU only for the setup of control data structures



More: Other Languages/Models

OpenACC, OpenMP; Kokkos

- Directive-based GPU programming models work analogously to CUDA
- GPU-awareness via MPI configuration, no need to copyout or map(from)
- Using explicit device pointer necessary: host_data use_device/use_device_addr

```
#pragma acc host_data use_device( A )  
MPI_Sendrecv( A+iy_start*nx+ix_start, (ix_end-ix_start), MPI_REAL_TYPE, top, 0,  
              A+iy_end*nx+ix_start, (ix_end-ix_start), MPI_REAL_TYPE, bottom, 0,  
              MPI_COMM_WORLD, MPI_STATUS_IGNORE );  
}
```

- Advanced communication libraries can be used like any other library
- Kokkos similar: Use Kokkos::View and Kokkos::View::data() (see [Wiki](#))

```
Kokkos::View<double*> A("A", nx*ny);  
MPI_Send(A.data(), int(A.size()), MPI_DOUBLE, bottom_rank, 0, COMM_WORLD);
```

Python

- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`

Python

- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`
- Very pythonic (and versatile): `cuPyNumeric`

Python

- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`
- Very pythonic (and versatile): `cuPyNumeric`

```
import numpy as np
```

```
A = np.random.rand(N, N)  
x = np.zeros(A.shape[1])  
d = np.diag(A)
```

Python

- CUDA-awareness in MPI in Python available via mpi4py
- Higher level: CuPy, CuTe DSL, Numba
- Very pythonic (and versatile): cuPyNumeric

```
import cupynumeric as np
```

```
A = np.random.rand(N, N)  
x = np.zeros(A.shape[1])  
d = np.diag(A)
```

Python

- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`
- Very pythonic (and versatile): `cuPyNumeric`

```
import cupynumeric as np
```

```
A = np.random.rand(N, N)  
x = np.zeros(A.shape[1])  
d = np.diag(A)
```

- `cuNumeric` [2]: transparently accelerates / distributes Numpy (and others)
 - Acceleration: Numpy kernel implementations for single-core CPU, multi-core CPU (OpenMP), and GPU (via libraries)
 - Distribution: OpenMP or MPI (via GASNet)
 - Type / size of task pool determined at start time via launcher script

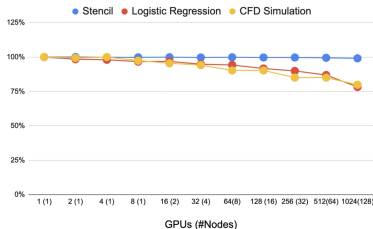
Python

- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`
- Very pythonic (and versatile): `cuPyNumeric`

```
import cupynumeric as np
```

```
A = np.random.rand(N, N)  
x = np.zeros(A.shape[1])  
d = np.diag(A)
```

- `cuNumeric` [2]: transparently accelerates / distributes Numpy (and others)
 - Acceleration: Numpy kernel implementations for single-core CPU, multi-core CPU (OpenMP), and GPU (via libraries)
 - Distribution: OpenMP or MPI (via GASNet)
 - Type / size of task pool determined at start time via launcher script



Python

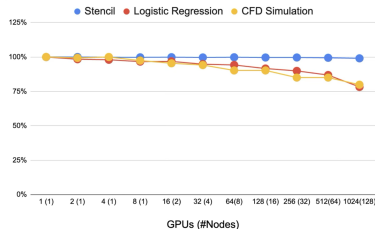
- CUDA-awareness in MPI in Python available via `mpi4py`
- Higher level: `CuPy`, `CuTe DSL`, `Numba`
- Very pythonic (and versatile): `cuPyNumeric`

```
import cupynumeric as np
```

```
A = np.random.rand(N, N)
x = np.zeros(A.shape[1])
d = np.diag(A)
```

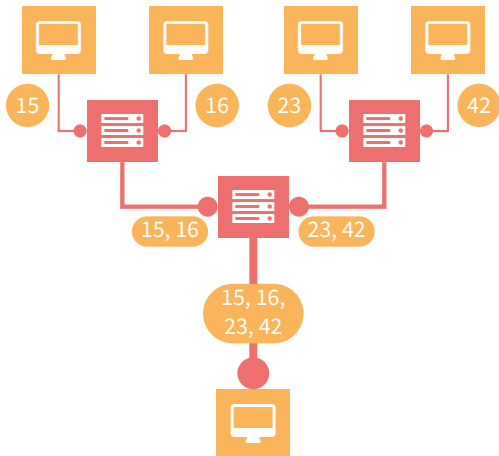
- `cuNumeric` [2]: transparently accelerates / distributes Numpy (and others)
 - Acceleration: Numpy kernel implementations for single-core CPU, multi-core CPU (OpenMP), and GPU (via libraries)
 - Distribution: OpenMP or MPI (via GASNet)
 - Type / size of task pool determined at start time via launcher script

→ <https://github.com/nv-legate/cunumeric/>



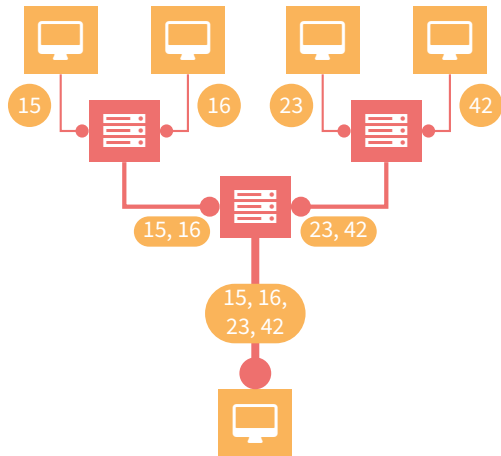
More: In-Network Computing

In-Network Computing

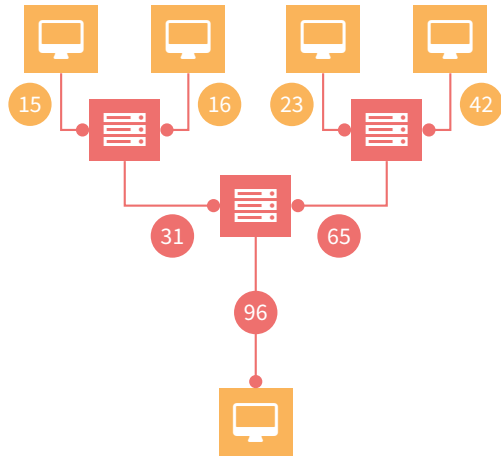


Traditional Reduce()

In-Network Computing



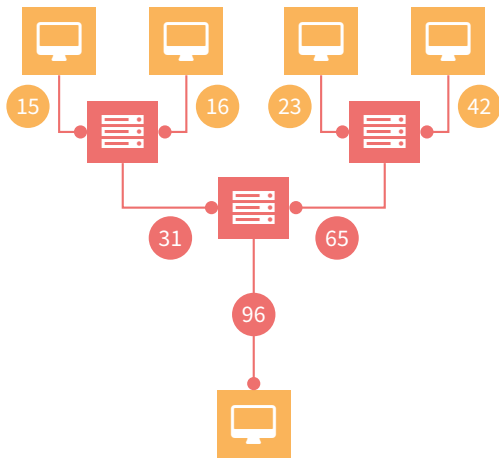
Traditional Reduce()



Switch-supported Reduce()

In-Network Computing

- Usually, network devices (switches, HCAs) just forward to computing devices
- Modern hardware offers in-network computation
- Works also with GPUs
- Less latency, less traffic
- Especially for communication-intensive collectives like `AllReduce()`



Switch-supported Reduce()

MPI MPI runtime transparently offloads specific collective operations to network, if enabled (OpenMPI, e.g. bundled in NVIDIA's HPC-X; MVAPICH2-X; also NCCL via plugin)

In-Network Computing Libraries

MPI MPI runtime transparently offloads specific collective operations to network, if enabled (OpenMPI, e.g. bundled in NVIDIA's HPC-X; MVAPICH2-X; also NCCL via plugin)

SHARP Lowest / base level API (*Scalable Hierarchical Aggregation and Reduction Protocol*)
libsharp_coll: interface, libsharp: backend



In-Network Computing Libraries

MPI MPI runtime transparently offloads specific collective operations to network, if enabled (OpenMPI, e.g. bundled in NVIDIA's HPC-X; MVAPICH2-X; also NCCL via plugin)

UCC New intermediate layer (from UCF initiative (UCX)) for *Unified Collective Communication*
→ <https://github.com/openucx/ucc>

SHARP Lowest / base level API (*Scalable Hierarchical Aggregation and Reduction Protocol*)
libsharp_coll: interface, libsharp: backend



In-Network Computing Libraries

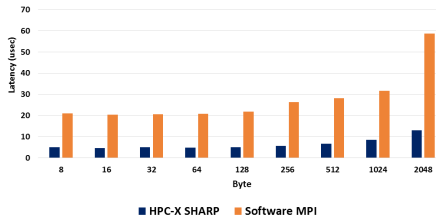
MPI MPI runtime transparently offloads specific collective operations to network, if enabled (OpenMPI, e.g. bundled in NVIDIA's HPC-X; MVAPICH2-X; also NCCL via plugin)

UCC New intermediate layer (from UCF initiative (UCX)) for *Unified Collective Communication*
→ <https://github.com/openucx/ucc>

SHARP Lowest / base level API (*Scalable Hierarchical Aggregation and Reduction Protocol*)
libsharp_coll: interface, libsharp: backend



MPI AllReduce Latency
1500 Nodes, 1PPN



Graph by Gil Bloch / Mellanox (2019)[9]

In-Network Computing Libraries

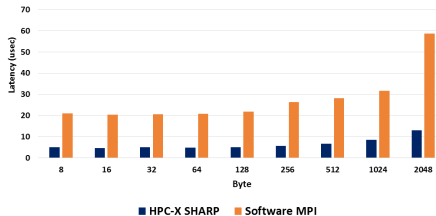
MPI MPI runtime transparently offloads specific collective operations to network, if enabled (OpenMPI, e.g. bundled in NVIDIA's HPC-X; MVAPICH2-X; also NCCL via plugin)

UCC New intermediate layer (from UCF initiative (UCX)) for *Unified Collective Communication*
→ <https://github.com/openucx/ucc>

SHARP Lowest / base level API (*Scalable Hierarchical Aggregation and Reduction Protocol*)
libsharp_coll: interface, libsharp: backend



MPI AllReduce Latency
1500 Nodes, 1PPN



Graph by Gil Bloch / Mellanox (2019)[9]

Other Vendors

AMD

- AMD Instinct GPUs entered HPC with a boom
- Multi-node ecosystem maturing rapidly
- Key technology already developed, mimicking NVIDIA's strategy
- UCX is ROCm enabled ([how-to](#) ); MVAPICH2-GDR [10] also optimized

Technology	NVIDIA	AMD
RDMA Support	GPUDirect RDMA	ROCmRDMA
Peer to Peer	GPUDirect P2P	ROCm IPC
Direct CPU Access (PCIe BAR)	GDRCopy BAR1	LargeBar
Accelerated Collectives	NCCL	RCCL
OpenSHMEM	NVSHMEM	ROC_SHMEM

AMD HIP Jacobi MPI Example

- Procedure: hipify-perl → fix errors → compile

- Code example

```
hipGetDeviceCount(&num_devices);  
hipSetDevice(local_rank%num_devices);  
real* a_ref_h;  
hipHostMalloc(&a_ref_h, nx * ny * sizeof(real));
```

- Compilation example

```
HIP_PLATFORM=amd hipcc --offload-arch=gfx90a -std=c++14 -munsafe-fp-atomics -O3 -fopenmp  
↪ -I${MPI_HOME}/include -c -o jacobi.cu.hip.o jacobi.cu.hip
```

```
HIP_PLATFORM=amd hipcc --offload-arch=gfx90a -std=c++14 -munsafe-fp-atomics -O3  
↪ -I${MPI_HOME}/include -L${MPI_HOME}/lib -lmpi --gcc-toolchain=${EBROOTGCCCORE} -o  
↪ jacobi.amd jacobi.cu.hip.o
```

- Needed: ROCm-aware UCX (UCX_TLS=rc_x,self,sm,rocm_copy,rocm_ipc)

Intel GPUs with SYCL

- SYCL: Native model for Intel GPU (also OpenMP); can also be executed on NVIDIA, AMD GPUs
- Very different programming model to CUDA, much more C++ish
- MPI supported as manual step
- Example: See [Codeplay's documentation](#)
- More SYCLic: Celerity, with distributed queues celerity.github.io/

```
queue q{property::queue::in_order()};  
q.submit([&](handler& h) {  
    h.parallel_for(num_items, [=](id<1> k) {  
        // jacobi_compute, fill result*  
    });  
});  
MPI_Sendrecv(&result, ..., );
```

Greatly reduced sketch of code based on [Intel documentation](#)

Summary, Conclusion

Summary, Conclusion

Efficient multi-node **GPU** computing is efficient multi-node computing with least possible amount of **CPU**

- Many advanced technologies and techniques in place to enable large-scale GPU applications
- GPU-aware MPI is key enabler
- On top / orthogonal: NCCL, NVSHMEM, ...→homework?
- *Appendix: [Links](#), [references](#)*

Summary, Conclusion

Efficient multi-node **GPU** computing is efficient multi-node computing with least possible amount of **CPU**

- Many advanced technologies and techniques in place to enable large scale applications
- GPU-aware MPI is key enabler
- On top / orthogonal: NCCL, NVSHMEM, ...→homework?
- *Appendix: [Links](#), [references](#)*

**Thank you
for your attention!**

p.chawla@fz-juelich.de

Appendix

References

Links I

- [1] *Support of GPU-aware MPI in mpi4py.* URL: <https://mpi4py.readthedocs.io/en/stable/overview.html#support-for-gpu-aware-mpi>.
- [3] *Legate (Numpy).* URL: <https://github.com/nv-legate/legate.numpy>.
- [4] *NVIDIA: HPC-X.* URL: <https://docs.mellanox.com/category/hpcx>.
- [5] *MVAPICH2.* URL: <https://mvapich.cse.ohio-state.edu/>.
- [6] *NVIDIA: NCCL SHARP Plugin.* URL: <https://github.com/Mellanox/nccl-rdma-sharp-plugins>.
- [7] *Unified Communication Framework (UCF) Consortium.* URL: <https://ucfconsortium.org/>.
- [8] *NVIDIA: SHARP.* URL: <https://docs.mellanox.com/category/mlnxsharp>.

References I

- [2] Michael Bauer and Michael Garland. “Legate NumPy: Accelerated and Distributed Array Computing.” In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '19. Denver, Colorado: Association for Computing Machinery, 2019. ISBN: 9781450362290. DOI: 10.1145/3295500.3356175. URL: <https://doi.org/10.1145/3295500.3356175> (pages 42–48).

- [9] Gil Bloch. “SHARP Tutorial.” In: *HPC Advisory Council 2019 Lugano Workshop*. 2019. URL: http://www.hpcadvisorycouncil.com/events/2019/swiss-workshop/pdf/020419/G_Bloch_Mellanox_SHARP_02042019.pdf (pages 53–57).

References II

- [10] Kawthar Shafie Khorassani et al. “Designing a ROCm-Aware MPI Library for AMD GPUs: Early Experiences.” In: *High Performance Computing*. Ed. by Bradford L. Chamberlain et al. Cham: Springer International Publishing, 2021, pp. 118–136. ISBN: 978-3-030-78713-4. URL: https://link.springer.com/chapter/10.1007%2F978-3-030-78713-4_7 (page 59).